Tyler Crossey 5/18/26
CSC230 Project 4 Pseudocode

SLL.h

Default constructor:
Make a reference to a headptr
Set size to 0

Destructor:
Walk and delete each node in linked list one by one

Make a temp node to reference current node (one being deleted)
Make a temp node to reference next node of current
Delete current node, then set it to next node, then set nextnode to next of current
Redo constructor in case we need to keep using the list

Insert:
Make a new node to add to the list
Set the ssn and name of it to the values provided by item1 and item2
Add node to the front of the sll (set its next to the headptr, then headptr to point to the new node)
Increment the size by 1

Search:
Make a temp node to use for walking and looking at values
Walk the list like in the destructor, but just with the current node temp node
Each loop check if the SSN matches the search value, if so return the pointer.

Remove:
Make a current and previous temp node like in destructor, walk with the same loop logic
However, rather than deleting each time we check if the current node has the called ssn
If it does, use the temp nodes to patch the hole (prev's next becomes curr next) and delete curr
Decrement size

Special cases are empty list (return false if head is null), head has key
(make curr point to old head, headptr point to next, delete curr)

hashTable.h

Constructor:
Same as default constructor but replace the hard coded 3 with a passed size variable

Note that table is an array of type sll

For each method that translates an input to a function in the hashing table, we do the hashing function to get the ssn's index (mod table size)

Find:
Hashing function
For the sll we get the index for, run search method for it and return true/false

Insert:
Hashing function
Search in sll to see if this ssn already exists there, if so fail to insert (return false)
For inserting, call that sll's insert method with the passed values and return true

Erase:
Hashing function
Run remove method for that sll and return its truth value (true for successful removal)

getSize:
Initialize a total variable to track total size
Go through each entry in the table array and add the size of the sll (using getSize defined in SLL.h)

Project4.cpp

Start the clock to track time (taken from project 3's timer code bit)

Create the hash table with parameter 10007 as described in project doc, type int
Make variables to track valid inserts, deletes, and retrieves

Repurposing the switch statement used in project 3, we have a char x to read what operation we do
A while loop assigns values from file input, we do i/d/r for insert/delete/retrieve respectively
We have a case for each that passes the ssn and increments the operation counter if it returns true

(note that for insertion we do not pass the name because it's not needed for what we are testing. it would be done by rewiring of the hashtable to accept string, and then convert the ssn from string to int when inserting.

End the clock as done in project 3

Output all our count variables as required